

# BuildOrbit Quality Assurance & Automation

## Summary

This document provides a comprehensive overview of the testing infrastructure, automated quality assurance workflows, and data seeding mechanisms implemented to guarantee the stability of the BuildOrbit enterprise platform.

---

### 1. Automated Testing Strategy (Playwright)

BuildOrbit enforces strict quality assurance through a robust, end-to-end (E2E) automation suite powered by **Playwright**. This suite simulates real user interactions against a live Chromium browser engine, bypassing isolated unit tests in favor of holistic workflow validation.

#### 1.1 The Comprehensive E2E Suite

The primary test matrix ( `tests/e2e/11-comprehensive.test.ts` ) is designed to validate the entire enterprise feature set sequentially.

- **Authentication & RBAC Isolation:** The setup script ( `auth.setup.ts` ) forcefully logs into five distinct browser contexts (SUPER\_ADMIN, ADMIN, HR, LEAD, ENGINEER) and caches their highly-secure `httpOnly` session cookies. The test suite then verifies that an ENGINEER cannot breach the HR Payroll routes, and an HR user cannot breach the CRM Invoice routes, ensuring watertight role-based access control.
- **HR & Workforce Workflows:** Automated tests simulate the complete employee lifecycle:
  1. Creating departmental Shifts.
  2. Executing dynamic UI selections to map Employees to Shifts.
  3. Simulating an Employee filing a Leave Request.
  4. Simulating the HR Admin approving said request and verifying the integer math for leave balance deductions.
  5. Generating monthly payroll drafts and triggering the secure PDF payslip download flow.
- **CRM Operations Workflows:** Automated tests simulate the sales pipeline:
  1. Creating a Sales Lead and entering required fields (Contact Person, Title).
  2. Transitioning the Lead state incrementally ( `NEW` → `CONVERTED` ).
  3. Generating a legally binding Invoice tied to the newly onboarded Client.

#### 1.2 Execution Commands

To execute the automated suite in an isolated CI/CD pipeline or local environment:

```
# Execute the entire suite silently (headless mode)
npx playwright test

# Execute the suite with a visual UI debugger (recommended for QA engineers)
npx playwright test --ui

# Target a specific module for rapid debugging
npx playwright test tests/e2e/05-leave.test.ts
```

---

## 2. Regression Prevention & Architectural Fortification

During the comprehensive technical audit, several static placeholders and architectural leaks were identified and fortified to prevent future regressions.

1. **Eradication of Hardcoded Fallbacks:** All dashboards (Admin, HR, and Employee) previously reliant on static mock arrays (e.g., `businessData` , `fakeLoginLogs` ) have been permanently refactored. The UI now strictly consumes real-time, aggregated data exclusively from the PostgreSQL relational database.
  2. **Server-Client Strict Boundaries:** React components have been aggressively audited. Previous critical leaks—such as passing Prisma client instances directly into browser-facing `"use client"` components within the Payroll module—have been structurally eliminated. Next.js App Router security paradigms are now strictly enforced.
- 

## 3. Deterministic Seed & QA Data Generation

A critical component of the QA strategy is the ability to instantly provision a realistic, fully-populated database environment. BuildOrbit ships with a highly deterministic seed script ( `prisma/seed.ts` ).

### 3.1 Seeding Operations

When executed, the seed script dynamically provisions:

- Core foundational data: Company Departments, Designations, and standardized Leave Types.
- A full hierarchy of authenticated users representing all 5 system roles.
- 14 days of realistic, randomized attendance history to populate the HR metrics dashboard.
- Accrued leave balances and historically generated payroll records.
- A populated CRM pipeline featuring active leads, closed deals, and chronological follow-up notes.

### 3.2 Resetting the Environment

QA Engineers and Developers can completely reset the application state to a pristine, test-ready configuration at any time using:

```
npx prisma db push --accept-data-loss
npx prisma db seed
```

*Warning: This will irreversibly destroy all current database contents and replace them with the deterministic seed state.*